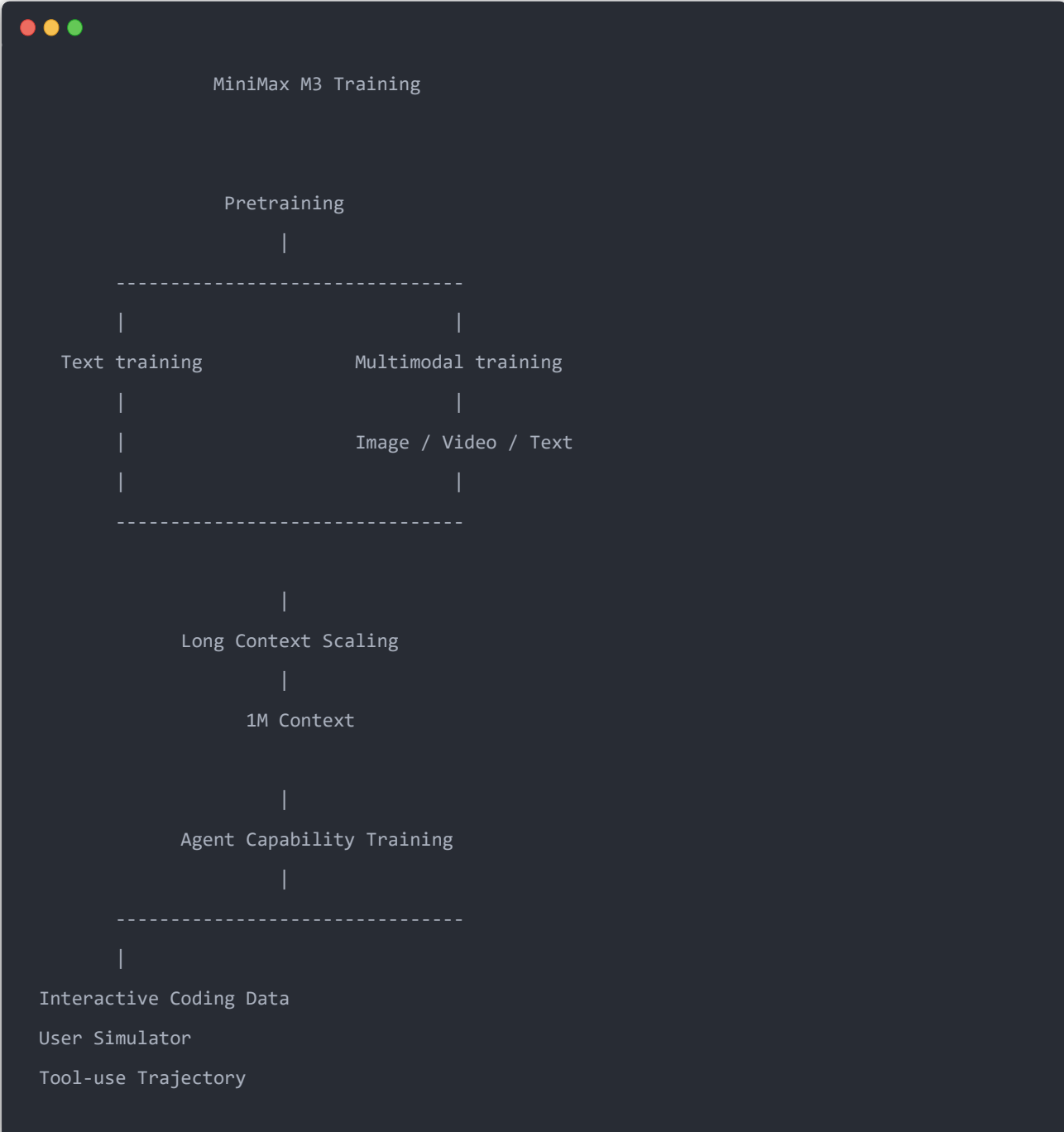


下面重点介绍 **MiniMax M3 的训练方法 (Training Pipeline)** 。需要先说明一点：目前这份 MiniMax M3 技术介绍文档主要披露了**训练思想、数据策略和能力培养方法**，没有公开完整的训练超参数（例如总 token 数、GPU 数量、学习率 schedule、batch size、RL 算法细节等）。因此下面严格基于文档内容展开。

如果对比你前面看的 GLM-5、DeepSeek-V4，M3 的训练重点明显不同：

- GLM-5：重点是 **Reasoning RL + Agent RL + 长上下文**
- DeepSeek-V4：重点是 **MoE scaling + MTP + 推理效率**
- MiniMax M3：重点是 **Agent 长程任务 + 真实交互数据 + 原生多模态 + 1M context**

整体训练路线：



1. M3 的核心训练目标: Agent First

MiniMax 明确表示:

M3 的训练目标之一是解决复杂 Agent 任务, 而最大挑战是 context scaling, 需要从 attention 机制底层解决。

所以它不是单纯追求:

benchmark QA

而是:

长期任务执行能力

例如:

一个真实开发任务:

用户需求
↓
阅读repo
↓
修改代码
↓

运行测试

↓

发现bug

↓

继续修改

↓

提交patch

整个过程可能产生：

几十万 token 历史。

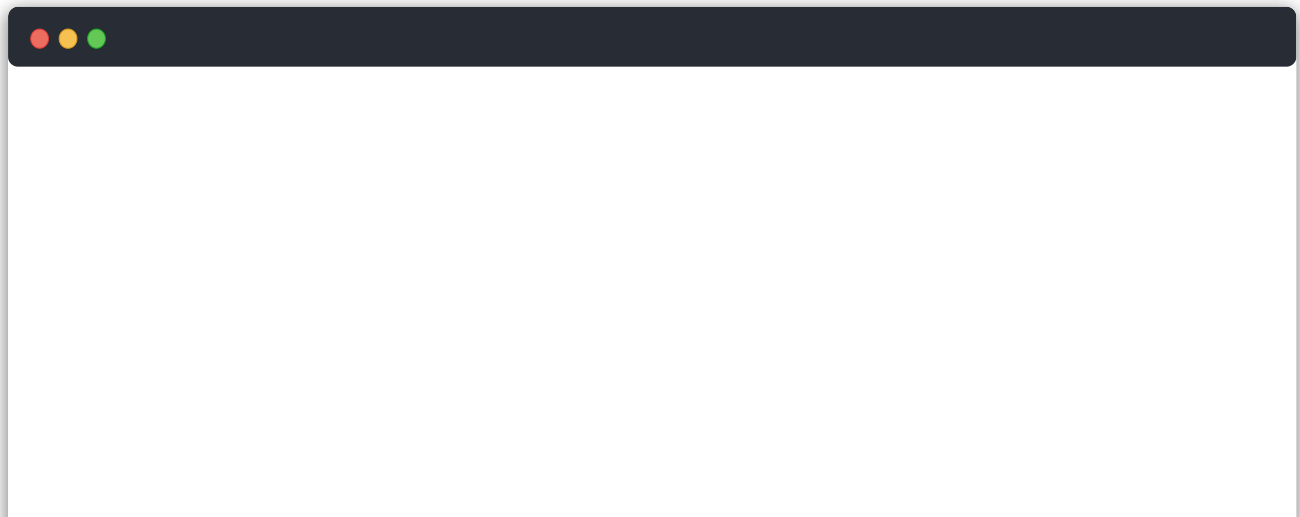
因此训练目标变成：

如何让模型在长时间、多轮交互中保持状态。

2. Pretraining: 从 Step 0 开始多模态训练

这是 M3 和很多 VLM 最大区别。

传统：



LLM



训练视觉adapter



得到多模态模型

例如：



LLaMA

+

CLIP

+

adapter

M3:



Step 0

Text

Image

Video

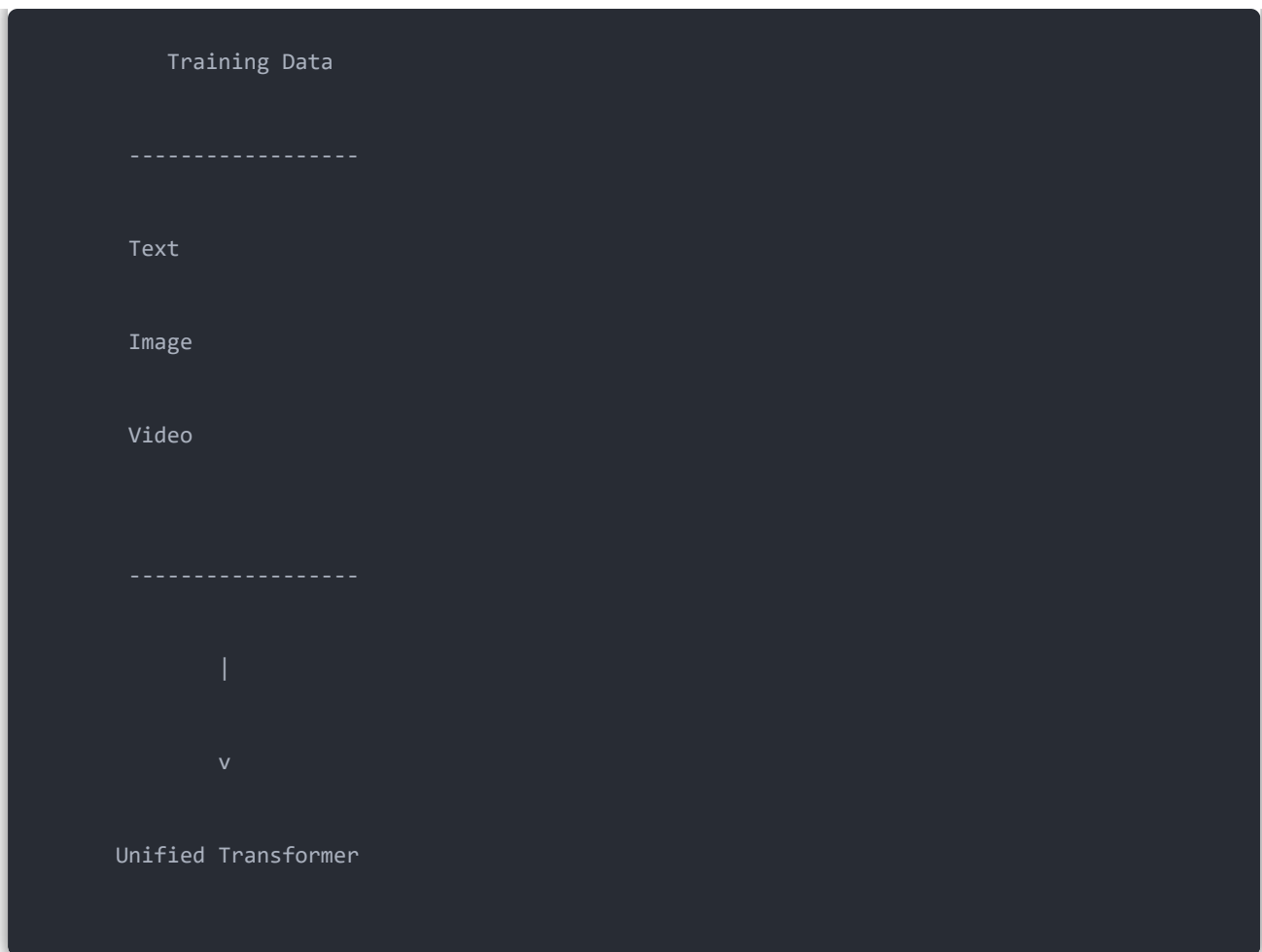
一起训练

论文：

M3 is a model that has undergone mixed-modality training from Step 0.

结构：





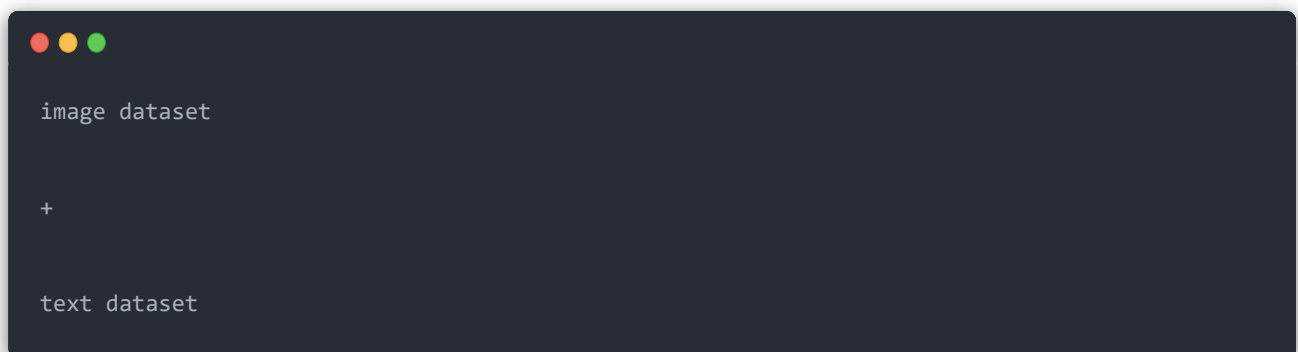
目的：

让不同模态共享同一个 semantic space。

3. Interleaved Multimodal Data

这是 M3 数据策略的重要创新。

不是：



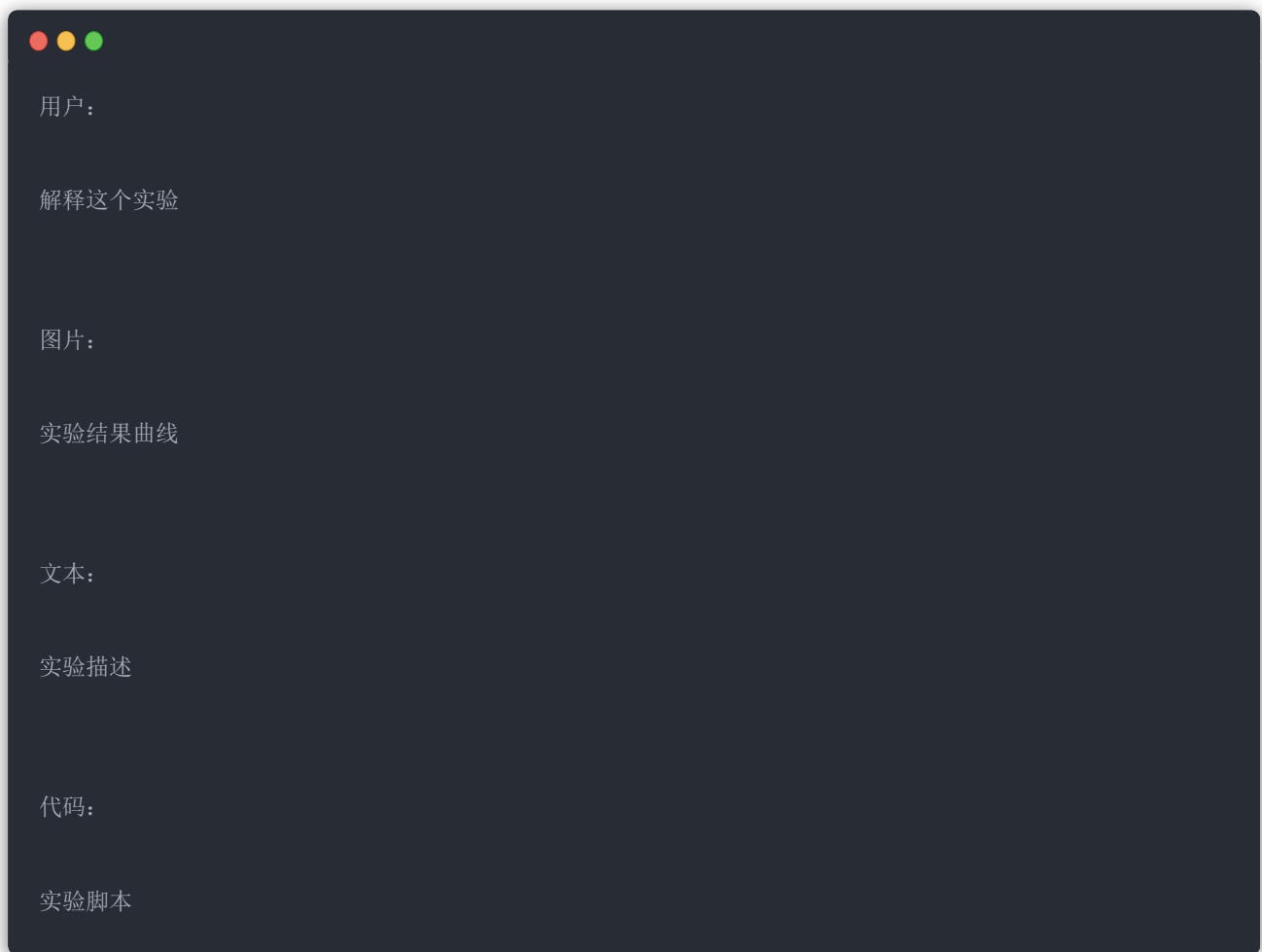
简单混合。

而是：

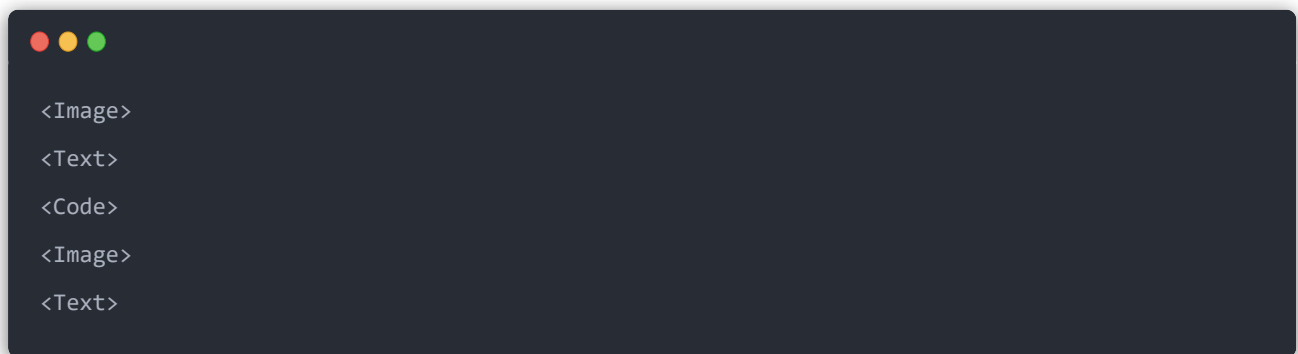
Interleaved Data

例如：

一个网页：



组成：



进入训练。

为什么？

因为真实 Agent 环境就是：



看到图片

理解文字

操作软件

修改代码

不是单模态任务。

论文指出：

interleaved data scales more easily than synthetic data.

4. Long Context Training: 为了 1M Context

M3 最大卖点：



1M tokens context

但关键不是简单把：



sequence length = 1M

打开。

训练必须解决：

Attention Scaling

传统：

[Attention(Q,K,V)

$\text{softmax}(QK^T)V]$

长度:

$[L=1M]$

计算:

$[10^{12}]$

不可接受。

所以 M3 训练阶段同步引入:



MSA Attention

5. MSA 训练流程

MSA不是推理阶段外挂。

需要训练模型学习:



哪些KV block重要

所以训练包含:

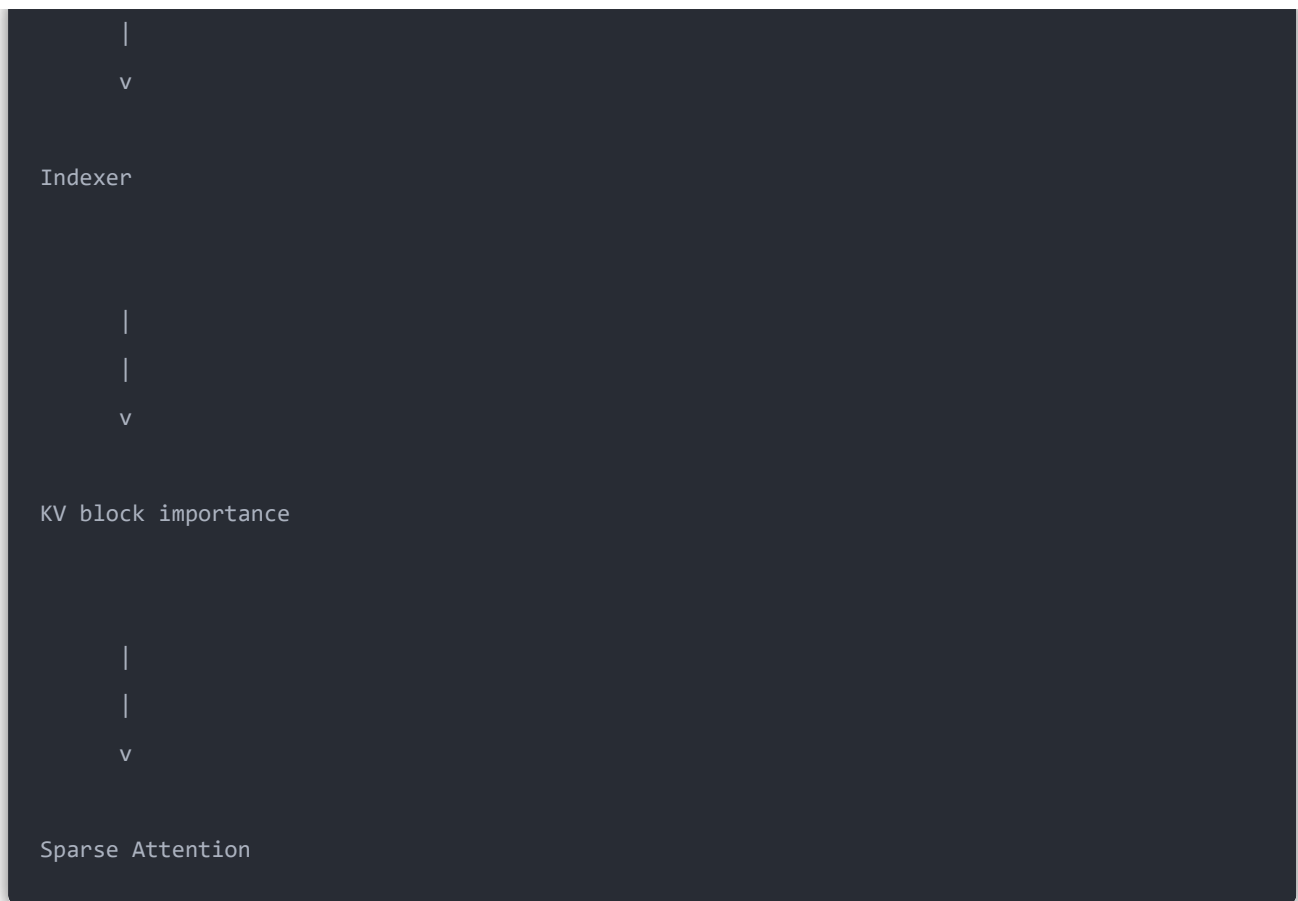
Indexer学习

整体:

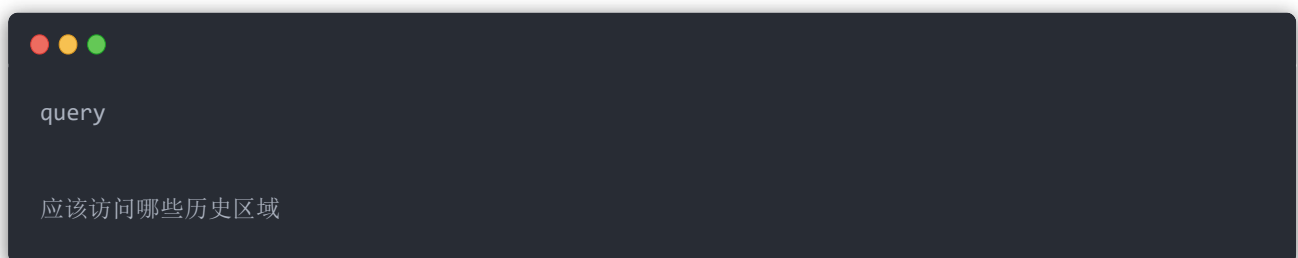


Hidden state

|



Indexer本质学习:



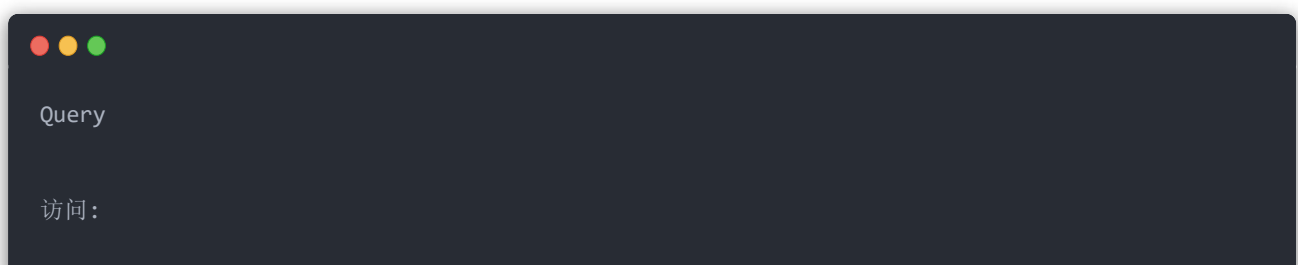
6. 为什么 M3 能保持 Sparse Attention 能力?

最大问题:

Sparse Attention容易丢信息。

例如:

Full Attention:



```
token1
token2
...
token1000000
```

MSA:

```
Query

只访问:

block 23
block 91
block 500
```

如果选错:

模型能力下降。

所以训练必须优化:

```
selection accuracy

+
attention quality
```

论文指出:

MSA 在大量 ablation 中:

matched full attention on the vast majority of capabilities.

7. Coding Agent 数据训练 (重点)

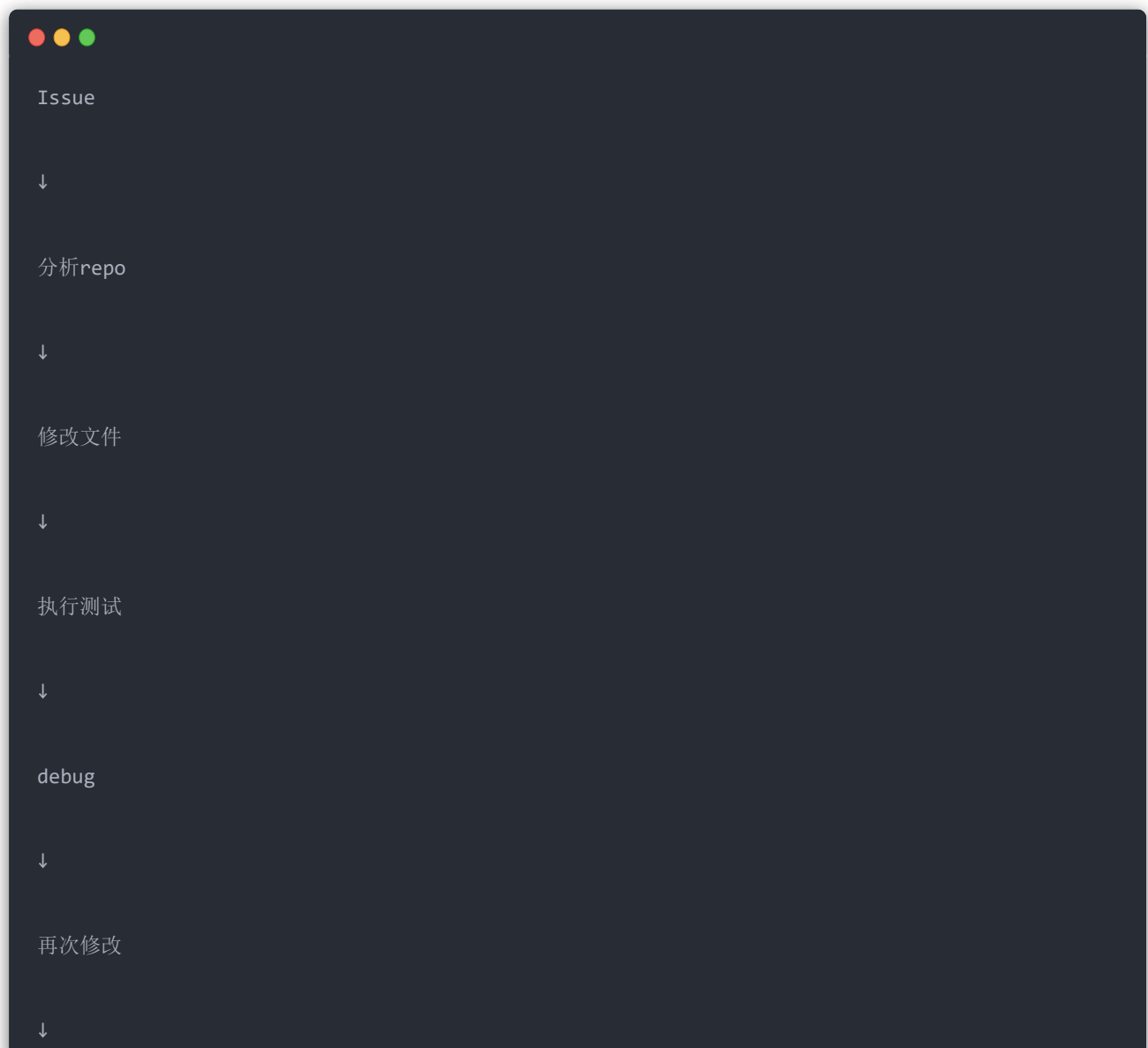
M3 的代码能力提升，不只是代码补全。

传统：



M3:

训练：



最终patch

也就是：

Long-horizon trajectory training

8. Interactive User Simulator

这是 M3 一个很特别的数据构造方式。

论文指出：

很多 coding benchmark：

假设：



single-turn task

但是现实：



用户不断交流

例如：

第一轮：



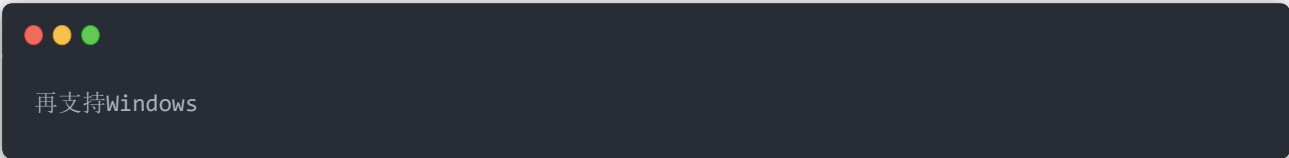
帮我优化性能

第二轮：



这里有bug

第三轮：



第四轮：



因此 M3 构建：

Interactive User Simulator

模拟真实开发者行为：

包括：

- 需求澄清
- 方案讨论
- 反馈修正
- 任务切换
- 项目迭代

训练数据：



v

Training sample

让模型学会：

不是一次生成答案。

而是：

长期合作。

9. Agent RL / Execution Feedback

虽然文档没有公开具体 RL 算法，但是可以看到 M3 大量依赖：

Environment Feedback

例如 CUDA Kernel 优化任务：

输入：



任务描述

benchmark script

Triton skeleton

模型：



生成kernel



运行benchmark

↓

获得性能

↓

继续优化

这是典型：



Agent + Environment loop

论文中的 CUDA kernel 案例：

M3：

24小时持续执行：

- 147次 benchmark submission
- 1959 次 tool call

最终：

FP8 GEMM：



7.6% peak utilization

↓

71.3%

提升：

9.4×

这说明训练重点不是：

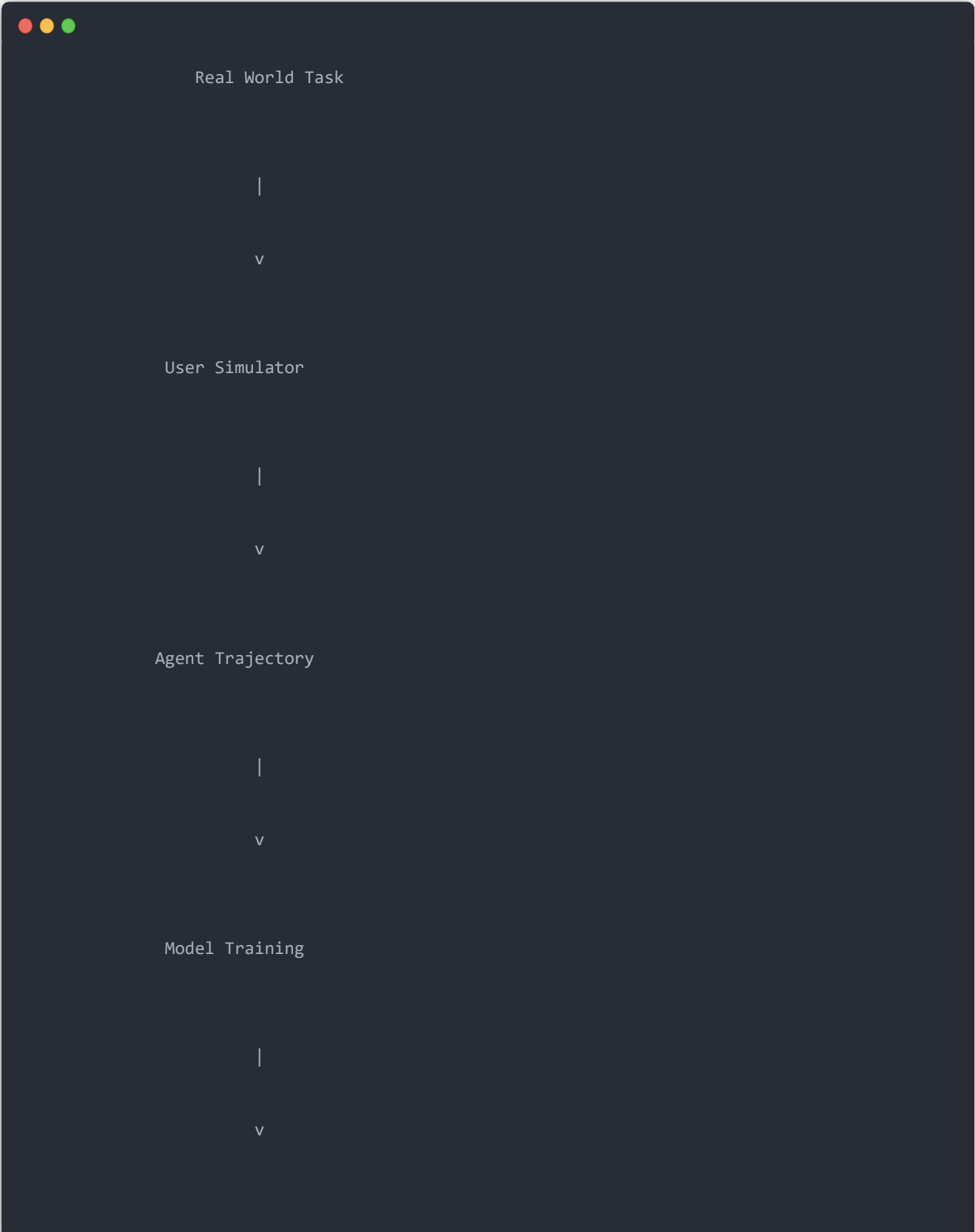
“生成代码”

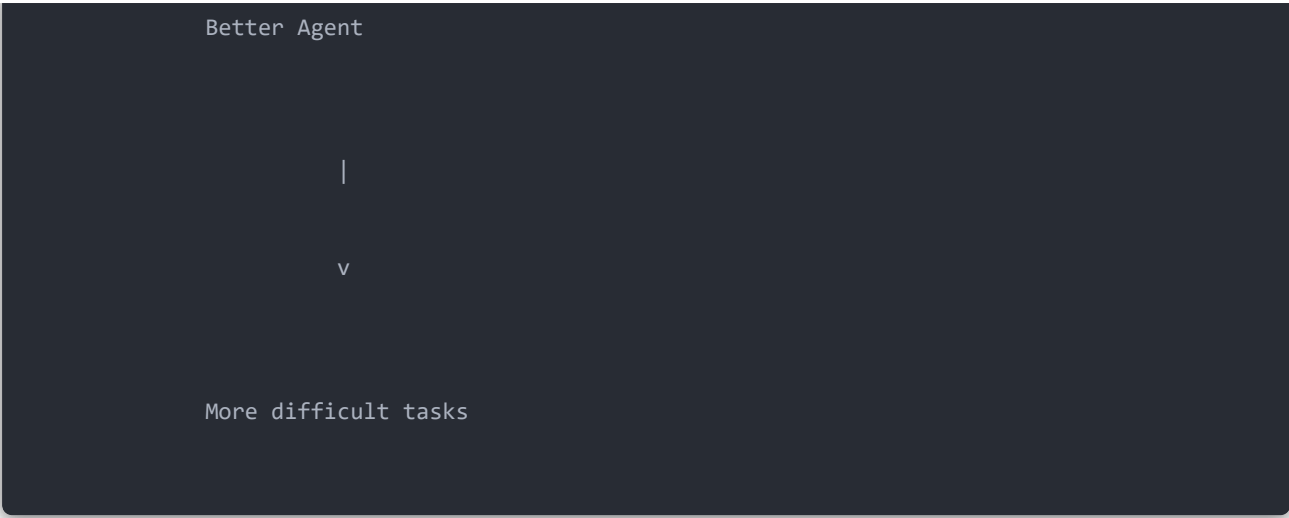
而是：

“通过反馈不断优化”。

10. M3 的训练数据闭环

可以总结成：





这是 Agent 模型和普通 LLM 最大区别。

11. M3 与 DeepSeek-V4 / GLM-5 训练对比

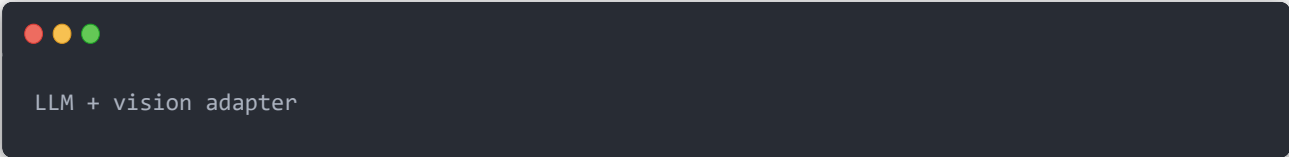
	DeepSeek-V4	GLM-5	MiniMax M3
Pretrain	大规模MoE	大规模MoE	多模态融合
Context	长上下文	200K	1M
Attention训练	MLA/DSA	DSA	MSA
代码数据	代码	Agent trajectory	真实交互
RL重点	推理	Reasoning+Agent	长期Agent
数据形式	文本	文本+轨迹	交互环境
目标	模型智能	推理Agent	自主执行

12. 最核心的训练创新总结

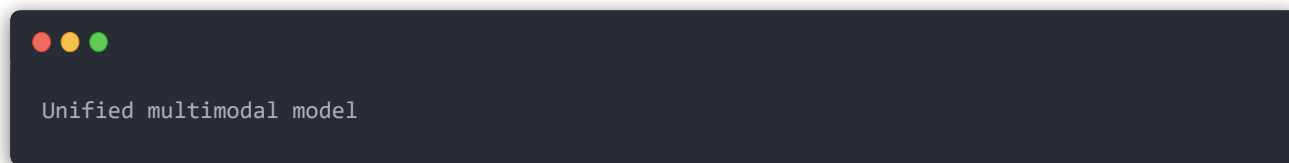
我认为 M3 训练最值得关注的是三个：

① 从 Step 0 的多模态训练

不是：

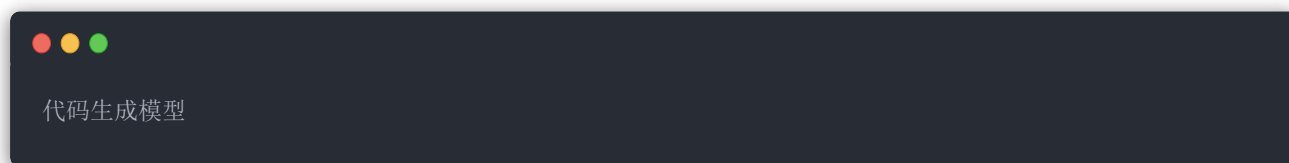


而是：

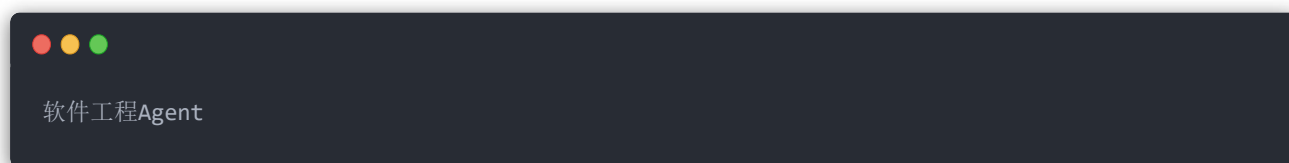


② Interactive Agent Trajectory Training

从：

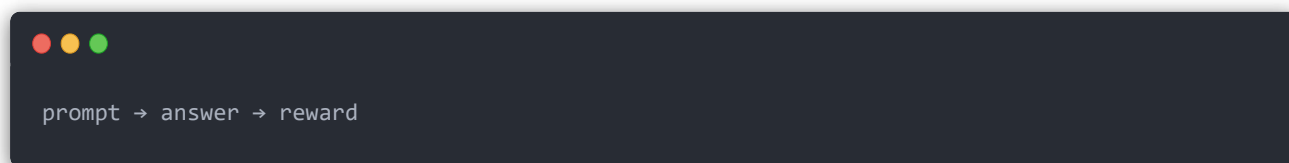


变成：



③ Long Horizon Reinforcement Learning

不是：



而是：



任务

↓

几十轮tool interaction

↓

最终结果reward

一句话总结：

MiniMax M3 的训练路线本质是围绕“长期自主 Agent”设计：从预训练阶段融合文本/图像/视频，到利用 interleaved 数据建立统一语义空间，再通过 MSA 支撑百万 token 上下文，最后利用真实开发者交互轨迹和环境反馈训练模型完成长周期任务。它训练的不是一个回答问题的 LLM，而是一个能够持续执行复杂工程任务的 Agent。